香港城市大學
City University of Hong Kong
Innovating into the Future

Chapter 6: Control

Control

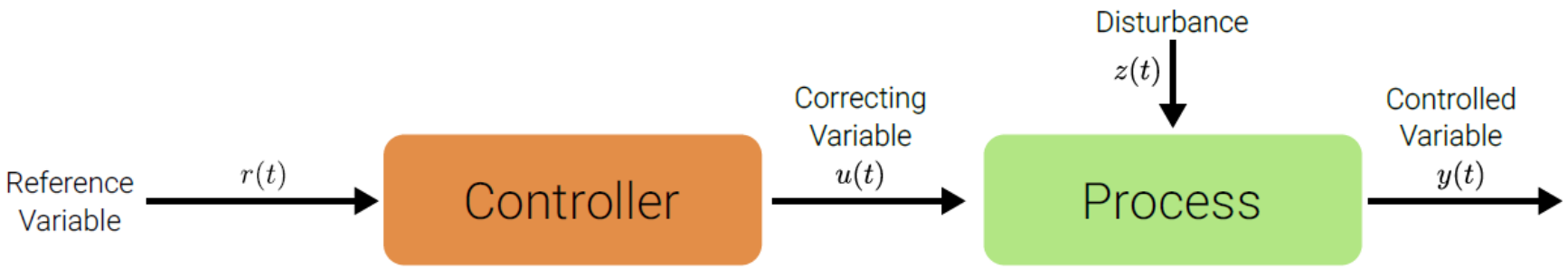
- Objective: Track the motion planner's trajectory by issuing commands to the steering, throttle, and braking actuators.

- Constraints: Actuation constraints, system constraints (tracking error limits), ride quality constraints

- Metrics: Tracking errors - speed, position (longitudinal/lateral), yaw

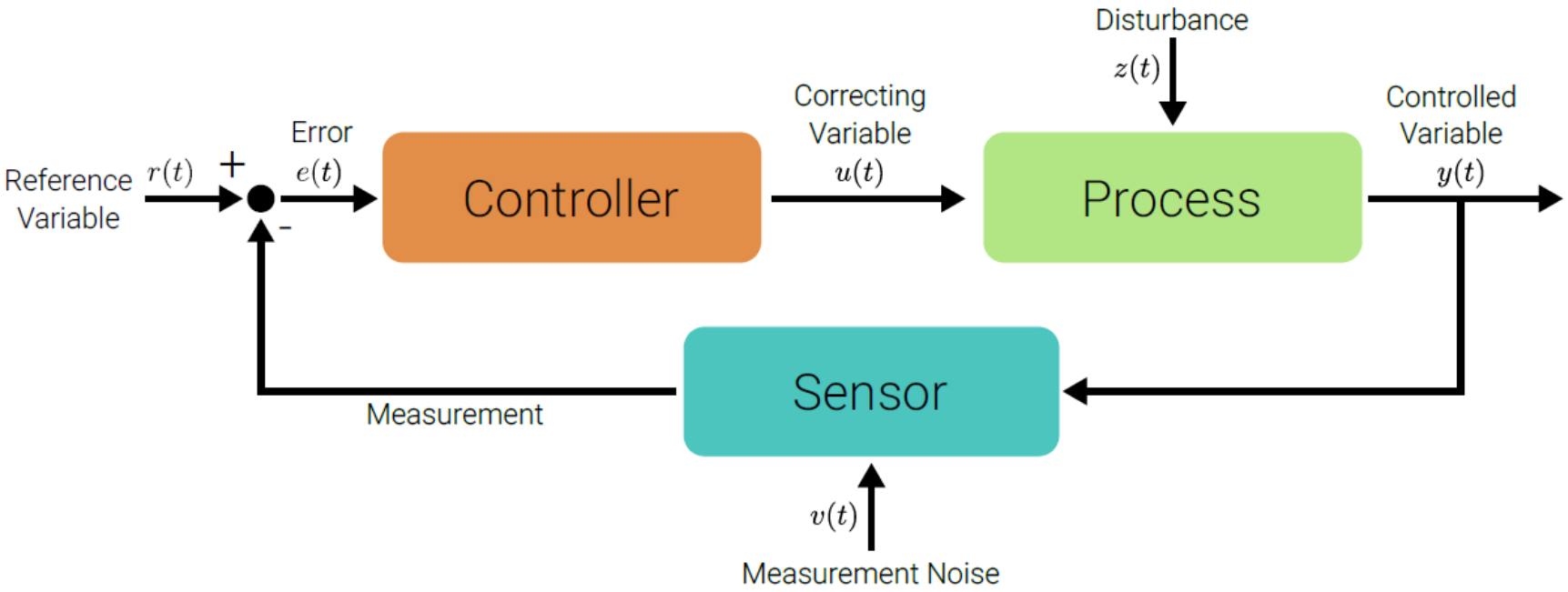
Open-Loop Control

- Requires **precise knowledge** of the plant and the influence factors
- **No feedback** about the controlled variable
- Cannot handle unknown disturbances, resulting in **drift**



Closed-Loop Control

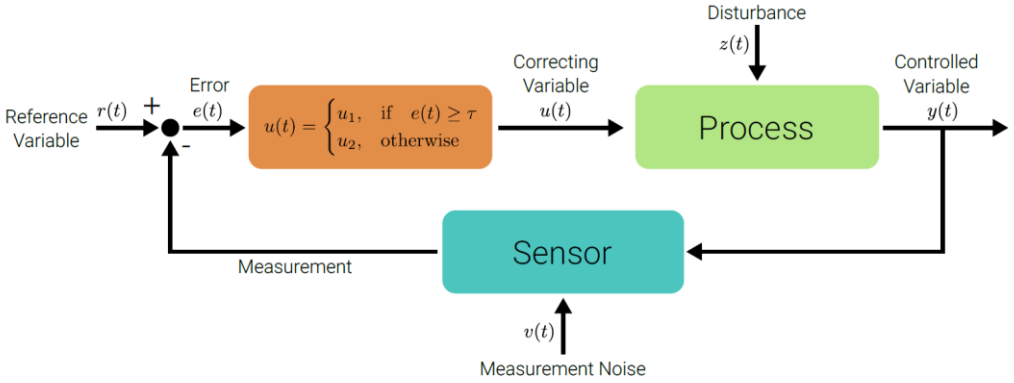
- Exploit feedback loop to minimize error between reference and measurement



Bang-Bang Control

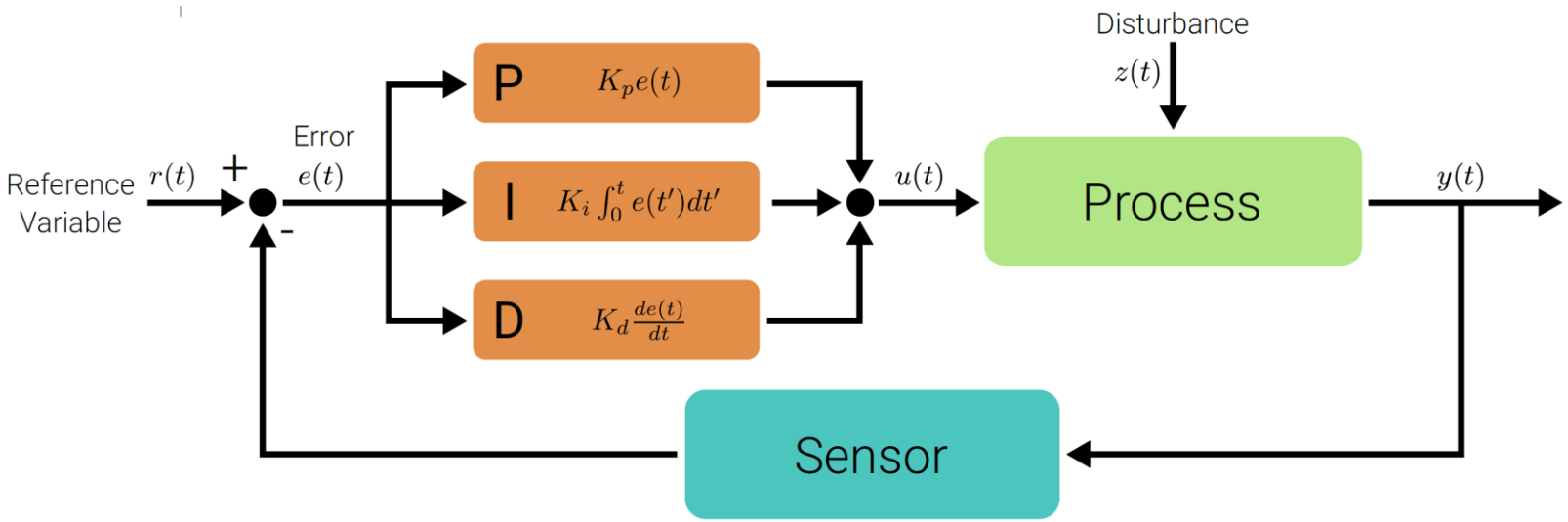
- Also called: hysteresis controller
- Often applied, e.g. in household thermostats
- Switches abruptly between two states
- Mathematical formulation:

$$u(t) = \begin{cases} u_1, & \text{if} \quad e(t) \geq \tau \\ u_2, & \text{otherwise} \end{cases}$$



PID Control

- Most common controller in current industrial applications
- Doesn’t require knowledge about plant/process to be controlled
- Comprises proportional (**P**), integration (**I**) and differential (**D**) element
- Reference variable r(t), correcting variable u(t), controlled variable y(t), error e(t)

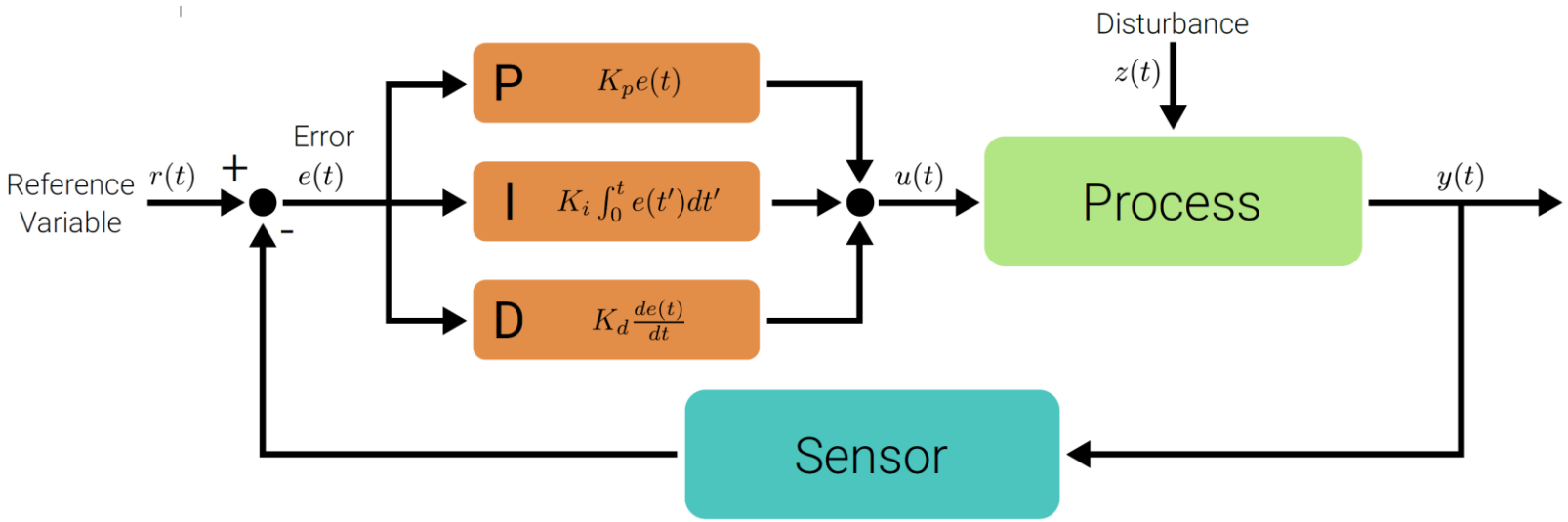


PID Control

- Mathematical formulation:

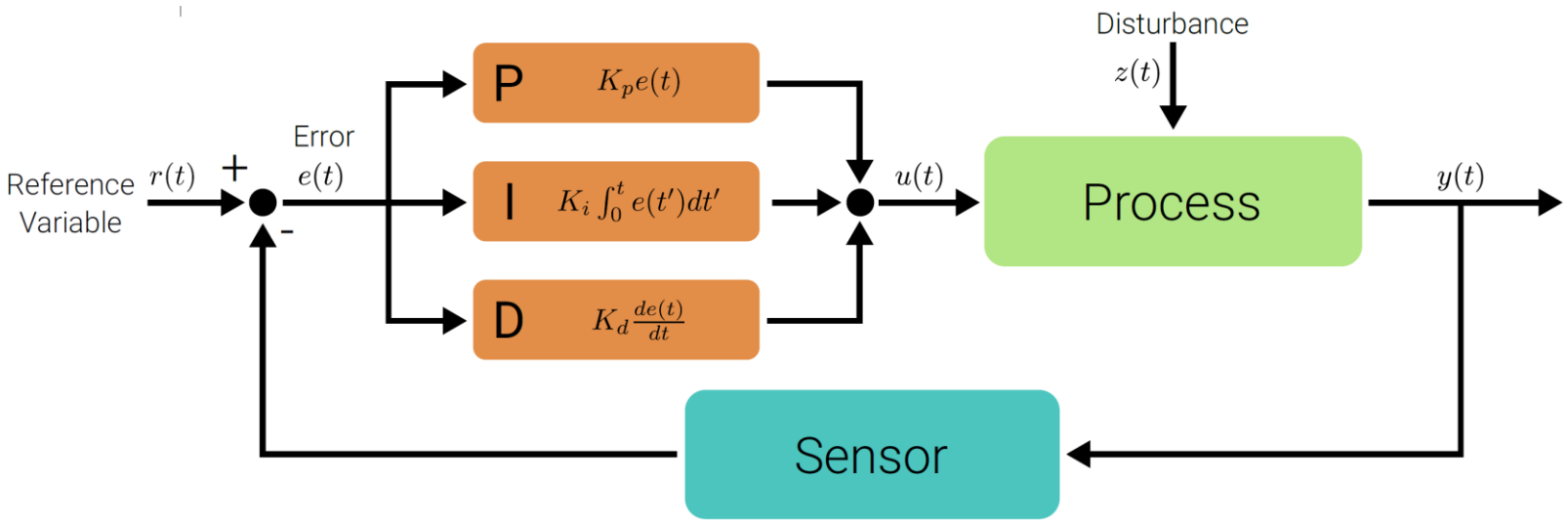
$$u(t) = K_p\, e(t) + K_i \int_0^t e(t')dt' + K_d \frac{de(t)}{dt}$$

with parameters K_p, K_i and K_d

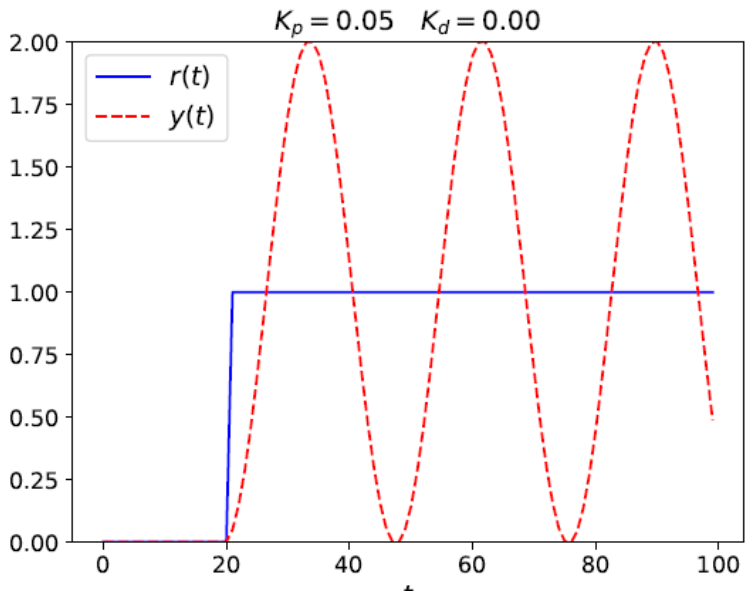


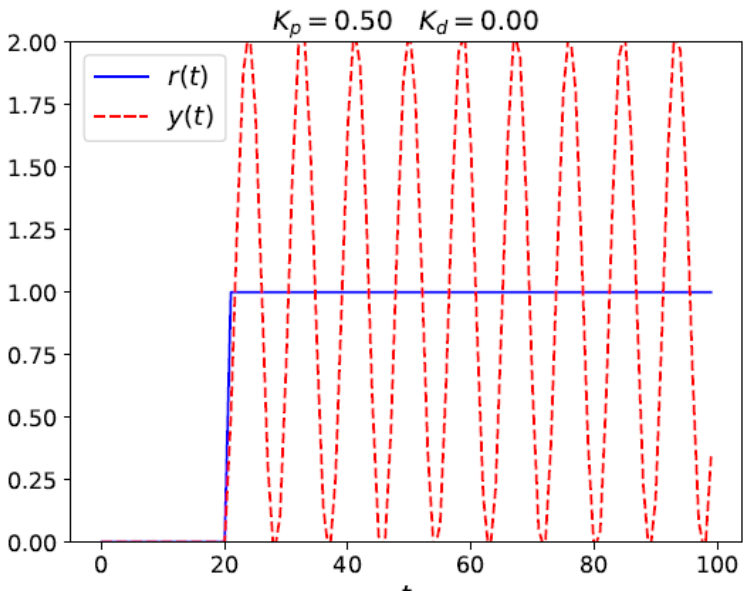
PID Control

- Using the **P** element alone leads to overshooting / oscillation
- Adding a **D** element alleviates this problem by introducing damping behavior
- The **I** element corrects residual errors by integrating past error measurements



P Control

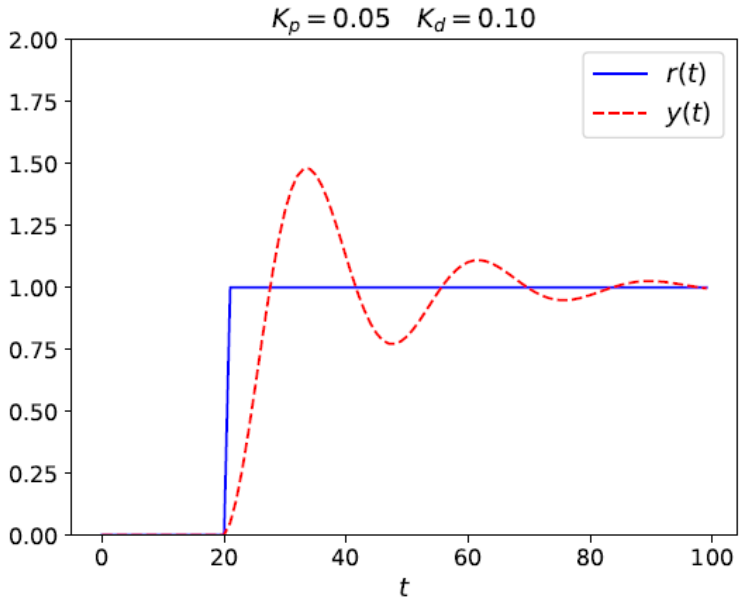


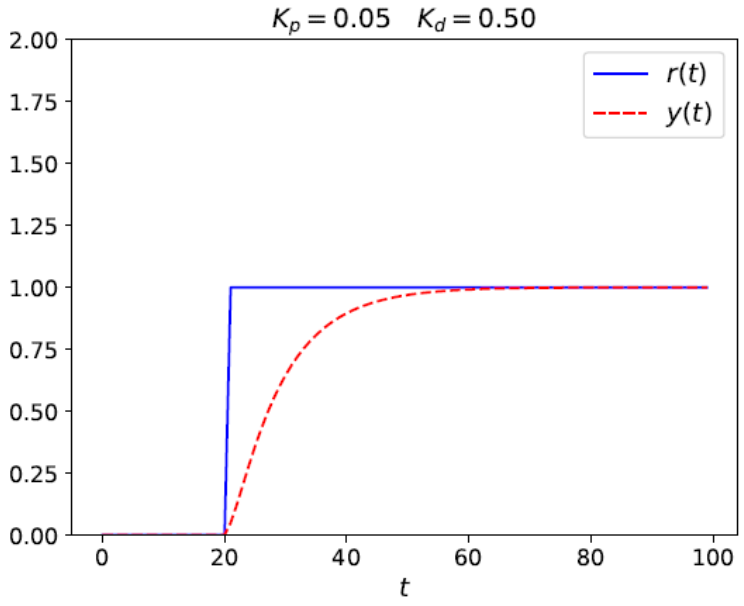


Controlled Variable: Position $y(t) = x(t)$

Correcting Variable: Acceleration $u(t) = a(t) = \ddot{x}(t)$

PD Control





Controlled Variable: Position $y(t) = x(t)$

Correcting Variable: Acceleration $u(t) = a(t) = \ddot{x}(t)$

PID Control

- As PID controllers are often used to control black box processes, various heuristics have been proposed for setting the parameters.
- **Ziegler-Nichols** is a commonly used heuristic

> - Set $K_i = K_d = 0$
> - Increase K_p until the ultimate gain $K_p = K_u$ where the system oscillates
> - Measure the oscillation period T_u at K_u
> - Set $K_p = 0.6K_u$, $K_i = 1.2K_u/T_u$ and $K_d = 3K_uT_u/40$

- These heuristics have been derived empirically by simulations
- While they provide a good starting point, manual fine-tuning is often required in practice.

PID Control

Example

- The ADS uses a discrete-position PID controller to adjust the throttle opening to achieve stable following of the target vehicle speed. The output u(k) is

$$u(k) = K_P \cdot e(k) + K_I \cdot \sum_{i=0}^{k} e(i) + K_D \cdot [e(k) - e(k-1)]$$

- Parameters
  - v_{target}=60 km/h
  - PID controller calibration parameters: $K_P = 2, K_I = 0.5, K_D = 0.2$
  - Controller sampling period T_s= 1 s
  - For simplifying computation, the $e(-1)$ and $\sum_{i=0}^{-1} e(i)$=0

PID Control

Example

- The ADS uses a discrete-position PID controller to adjust the throttle opening to achieve stable following of the target vehicle speed. The output u(k) is

$$u(k) = K_P \cdot e(k) + K_I \cdot \sum_{i=0}^{k} e(i) + K_D \cdot [e(k) - e(k-1)]$$

- Parameters
  - v_{target}=60 km/h
  - PID controller calibration parameters: $K_P = 2, K_I = 0.5, K_D = 0.2$
  - Controller sampling period T_s= 1 s
  - For simplifying computation, the $e(-1)$ and $\sum_{i=0}^{-1} e(i)$=0

PID Control

Q1: At time k=0 (initial sampling time), the vehicle's onboard sensor measures an actual vehicle speed v(0) = 58 km/h. If under proportional control only, the controller output throttle opening u(0) at k=0 is?

Answer:

- e(0)=60-58=2 km/h
- u(0)=2*2=4

PID Control

Q2: If the controller is switched to PI control. The known timing data is as follows:

- v(0) = 55 km/h
- v(1) = 58 km/h
- v(2) = 60 km/h.

Calculate the controller output throttle opening u(1) at k=1 under PI control.

Answer:

e(0)=5,e(1)=2, $\sum_{i=0}^{1} e(i)$=7; u(1)=2*2 + 0.5*7=7.5

Q3: At time k=3, the vehicle enters a long uphill section, and its speed drops significantly. The controller activates full PID control, and the throttle opening strictly adheres to the engineering limit of 0% to 100%. We have

- $\sum_{i=0}^{2} e(i)$ = 5;
- v(2) = 60 km/h
- v(3) = 20 km/h

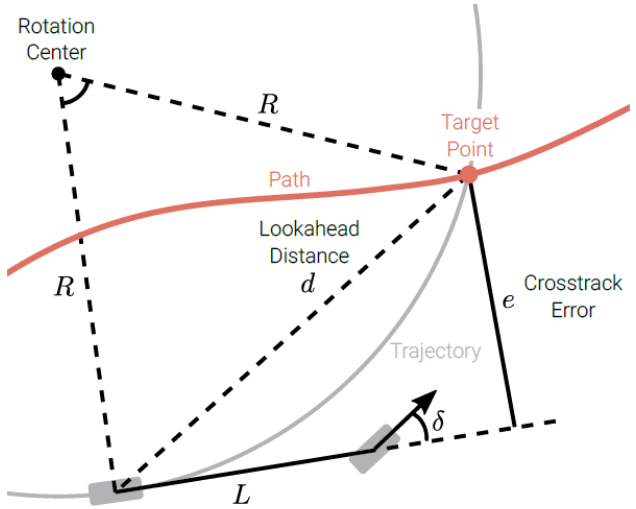
Please calculate (1) the theoretical output of the controller at k=3, and (2) the actual throttle output at k=3 .

Answer: e(2)=0, e(3)=40, $\sum_{i=0}^{3} e(i) = 5 + 40 = 45$,

u=2*40+0.5*45+0.2*(40-0)=110/.5

Pure Pursuit Control

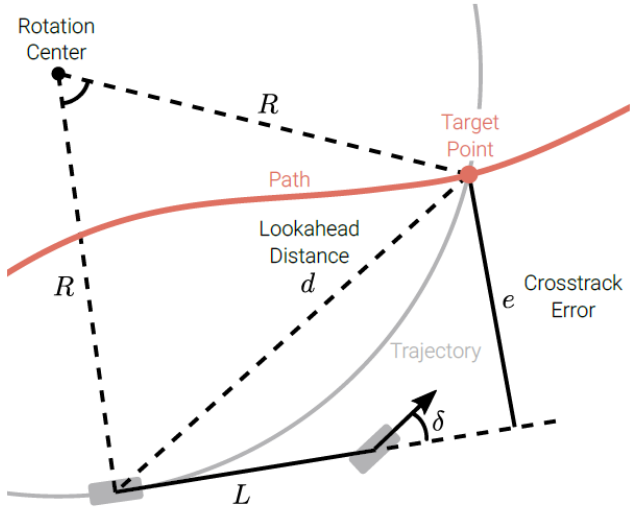
- Simulates the "human driving" logic—looking ahead to a target point on the path and adjusting the steering angle to approach the target, achieving smooth and accurate path following.
- Takes the "look-ahead distance" as a key parameter, which determines the responsiveness and stability of the control system
- Unlike simple position feedback control, it focuses on the future path trend, avoiding frequent steering and improving tracking smoothness.



Pure Pursuit Control

Core Principle

- Step 1: Predefine a reference path (trajectory).
- Step 2: Determine a “target point" on the reference path
  - this point is at a fixed "look-ahead distance" from the current position of the vehicle/robot.
- Step 3: Calculate the required steering angle by geometric relationship (between the current position, look-ahead point, and vehicle/robot heading), so that the vehicle/robot moves towards the look-ahead point.
- Step 4: Real-time update the current position and look-ahead point, dynamically adjust the steering angle, and maintain path following.

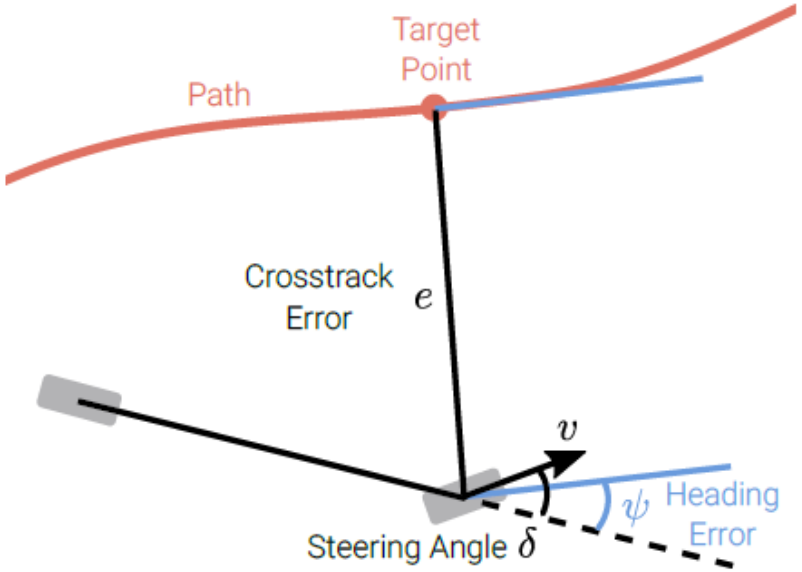


Stanley Control

- **Control Law** used by **Stanley** in **DARPA Challenge**

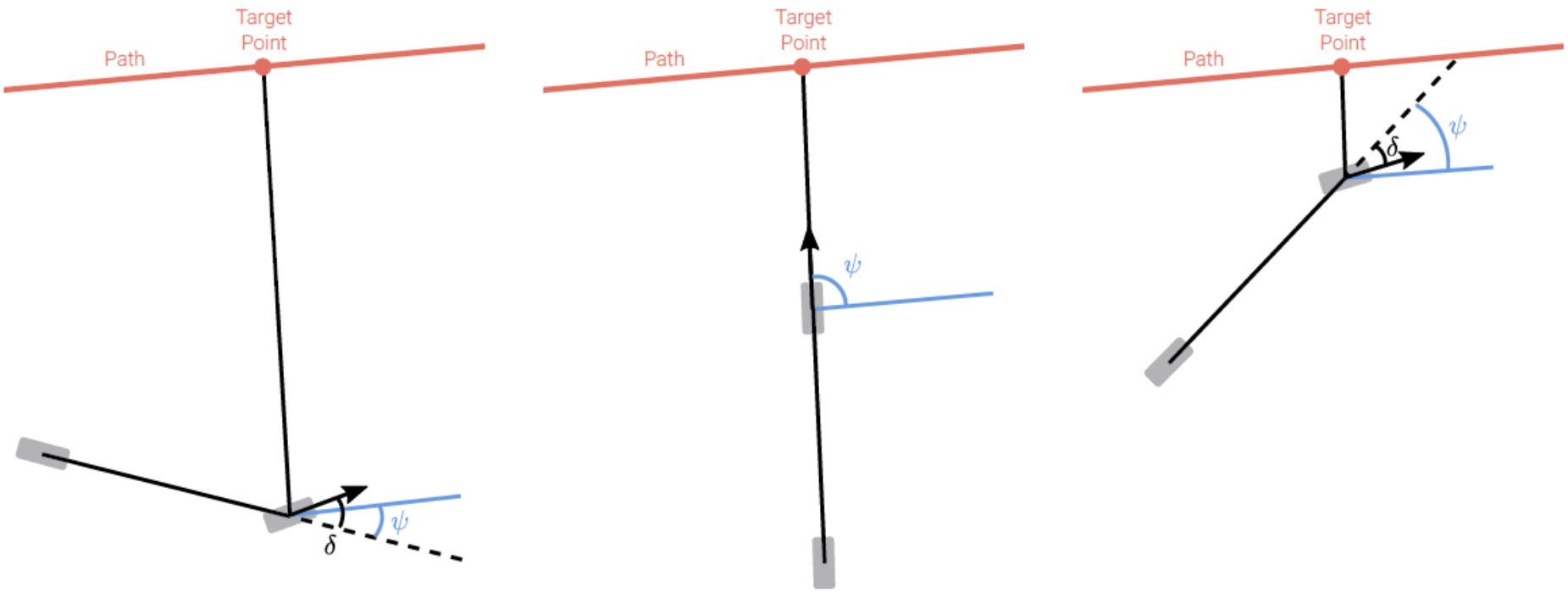
$$\delta = \psi + \tan^{-1}\left(\frac{k\,e}{v}\right)$$

- v = speed, ψ = heading err., e = crosstrack err.
- Reference at **front axle, no lookahead**
- Combines **heading** and **crosstrack error**
- It can be shown that the crosstrack error **converges** exponentially to 0 (indep. of v)
- Works for small velocities w/o disturbances



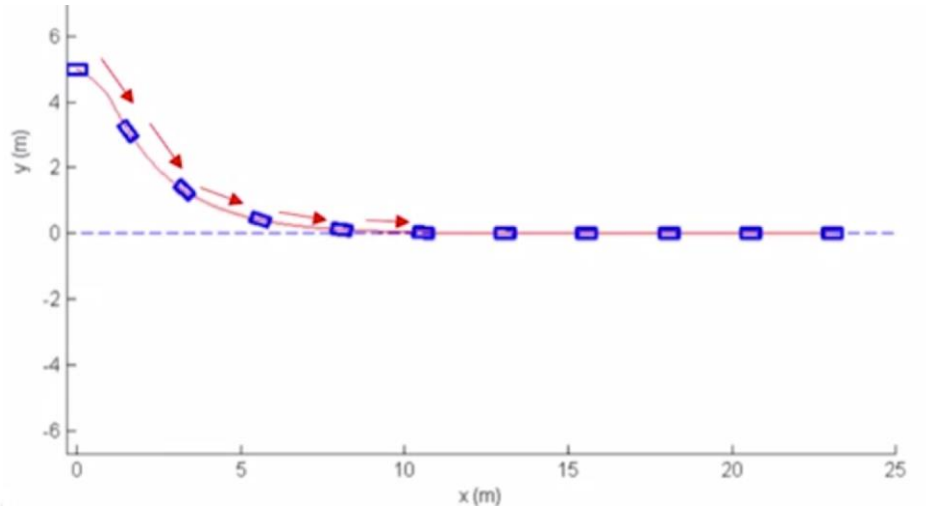
Stanley Control

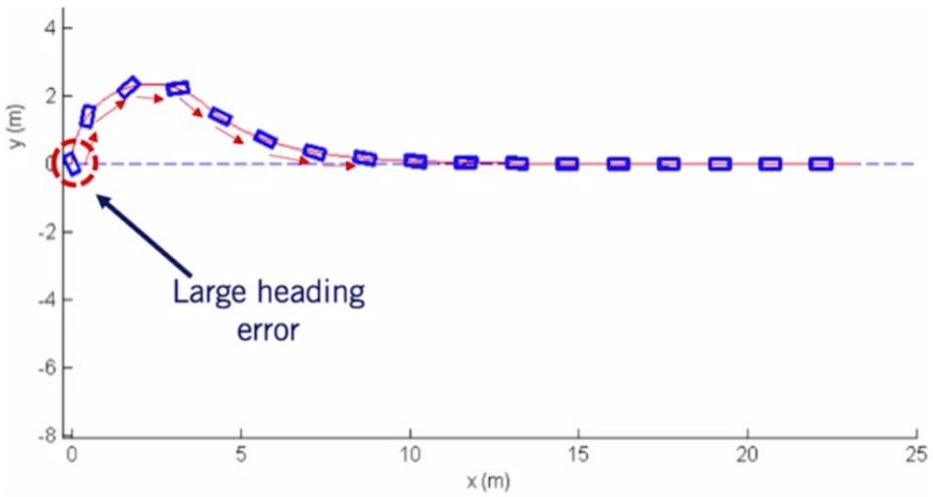
- As heading changes, heading correction **counteracts** crosstrack correction



Stanley Control

- The Stanley controller can correct **large crosstrack** and **large heading errors**
- **Global stability:** Independent of initial conditions guides vehicle back (proven)
- But does not consider noisy observations, actuator dynamics, tire force effects
- Softening/dampening terms and curvature information can be added





Linear Quadratic Regulator (LQR)

- LQR is an **optimal controller** for **linear systems**
  - Given a system, it finds the best control action that stabilizes the system *with minimal cost*
- For the continuous-time linear system $\dot{x} = Ax + Bu$
- The quadratic cost functional defined as $J = \int_0^\infty \left(\mathbf{x}^T Q \mathbf{x} + \mathbf{u}^T R \mathbf{u}\right) dt$
  - Q: state cost → penalize how far you are from target

    Larger Q: track state more tightly (aggressive)
  - R: control cost → penalize how much you use the control

    Larger R: use control more gently (smooth, conservative)
- The optimal control is linear feedback $\mathbf{u} = -K\mathbf{x}$

$$K = R^{-1} B^T P$$

and *P* comes from solving the Algebraic Riccati Equation (ARE)

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

(solved by libraries)

Model Predictive Control

- LQR is optimal control over infinite time horizon for linear systems

- Model Predictive Control (MPC): iterative finite-horizon, receding-horizon optimal control method that works for both linear and nonlinear systems.
- At each time step, it solves a finite-horizon optimal control problem (typically over N future steps) and applies only the first control action, then repeats the optimization at the next step.
- **Can handle constraints** on states, inputs, and outputs
  - Big advantages for real-world systems (e.g., actuator saturation, safety limits).

Model Predictive Control

- For a linear system:

$$x_{k+1} = Ax_k + Bu_k$$

- At time step k, MPC minimizes a quadratic cost over a prediction horizon N:

$$J_k = \underbrace{x_N^T P x_N}_{\text{terminal cost}} + \sum_{i=0}^{N-1} \underbrace{(x_i^T Q x_i + u_i^T R u_i)}_{\text{stage cost}}$$

  - Q: state cost → penalize how far you are from target
  - R: control cost → penalize how much you use the control
  - P≻0: terminal penalty matrix (ensures stability, often the solution to LQR's Algebraic Riccati Equation)

- Constraints

$$x_{\min} \leq x_i \leq x_{\max}, \quad i = 0, 1, \ldots, N$$ state bounds (e.g., safety limits)

$$u_{\min} \leq u_i \leq u_{\max}, \quad i = 0, 1, \ldots, N-1$$ input bounds (e.g., actuator limits)

Model Predictive Control

- Optimal Control Problem: at each time step k, MPC solves the following constrained optimization problem:

$$
\begin{aligned}
&\min_{U_k} \quad J_k(x_k, U_k) \\
&\text{subject to} \\
&x_{i+1} = Ax_i + Bu_i, \quad i = 0, 1, \ldots, N-1 \\
&\quad x_0 = x_k \quad \text{(initial state)} \\
&x_{\min} \le x_i \le x_{\max}, \quad i = 0, 1, \ldots, N \\
&u_{\min} \le u_i \le u_{\max}, \quad i = 0, 1, \ldots, N-1
\end{aligned}
$$

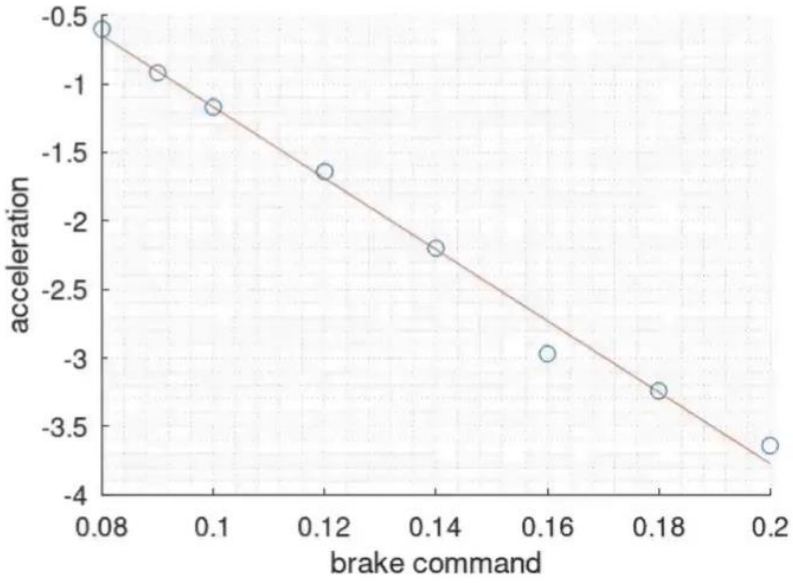
$$
J_k = \underbrace{x_N^T P x_N}_{\text{terminal cost}} + \sum_{i=0}^{N-1} \underbrace{(x_i^T Q x_i + u_i^T R u_i)}_{\text{stage cost}}
$$

  - Solving it is a **Quadratic Programming (QP)** problem, which requires numerical solvers.

- Receding Horizon Principle: After solving the problem, only the first control action is applied. The rest of the optimal sequence is discarded. At the next time step, the problem is re-solved with the new states

Model Predictive Control

- Strengths
  - Natively handles hard constraints (input, state, output) – main advantage.
  - Works for linear & nonlinear systems.
  - Receding horizon provides natural feedback & disturbance rejection.
- Limitations
  - Heavy online computation: must solve QP/NLP every time step
  - Stability not automatic – needs terminal cost/set.
  - More tuning parameters: N,Q,R,P, constraints.
  - Performance depends strongly on model quality.

Actuators

- Actuators need to be characterized to know the system's abilities and bounds
- Example of important limits:
  - Max acceleration per speed
  - Max deceleration
  - Relationship between steering angle and yaw rate
  - Maximum steering rate
  - Maximum/minimum jerk per actuator